# LeetCode Practice

## 12 Easy: Math

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

**12**
**EASY**

**Patterns (in order)**

1. Math  (12)

**How to use**

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has  < Prev  |  Contents  |  Next >  at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents  >

# Contents

**12 questions | tap a title to jump to it**


| Easy questions (12) | | Go > |
|---|---|---|
| **Math (12)** | | **Go >** |
| E | 1. Palindrome Number | LC 9 |
| E | 2. Roman to Integer | LC 13 |
| E | 3. Plus One | LC 66 |
| E | 4. Excel Sheet Column Title | LC 168 |
| E | 5. Ugly Number | LC 263 |
| E | 6. Base 7 | LC 504 |
| E | 7. Confusing Number | LC 1056 |
| E | 8. Greatest Common Divisor of Strings | LC 1071 |
| E | 9. Missing Number in Arithmetic Progression | LC 1228 |
| E | 10. Perform String Shifts | LC 1427 |
| E | 11. Number of Good Pairs | LC 1512 |
| E | 12. Time Needed to Buy Tickets | LC 2073 |

## Easy questions (12)

EASY | TOPIC 1 OF 1

# Math

**12 Easy questions start on the next page**

**Before this:** this is the first topic
**After this:** this is the last topic

---

**IN THIS TOPIC**

**1.** Palindrome Number LC 9
**2.** Roman to Integer LC 13
**3.** Plus One LC 66
**4.** Excel Sheet Column Title LC 168
**5.** Ugly Number LC 263
**6.** Base 7 LC 504
**7.** Confusing Number LC 1056
**8.** Greatest Common Divisor of Strings LC 1071
**9.** Missing Number in Arithmetic Progression LC 1228
**10.** Perform String Shifts LC 1427
**11.** Number of Good Pairs LC 1512
**12.** Time Needed to Buy Tickets LC 2073

EASY | Math | LeetCode #9 | 1 of 12 | 1/12

# Palindrome Number

## QUESTION

Given an integer x, return true if x is a palindrome, and false otherwise.

**Example 1**

| | |
|---|---|
| Input | `x = 121` |
| Output | `true` |
| Explanation | *121 reads as 121 from left to right and from right to left.* |

**Example 2**

| | |
|---|---|
| Input | `x = -121` |
| Output | `false` |
| Explanation | *From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.* |

**Example 3**

| | |
|---|---|
| Input | `x = 10` |
| Output | `false` |
| Explanation | *Reads 01 from right to left. Therefore it is not a palindrome.* |

CONSTRAINTS

- $2^{31} <= x <= 2^{31} - 1$
- Follow up: Could you solve it without converting the integer to a string?

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if an integer reads the same forwards and backwards.

Negative numbers are not palindromes. The follow-up asks to solve without string conversion. Right?"

"A few quick questions:

x=0 -> true? Yes — 0 reads the same. x=10 -> false? Yes — '10' reversed is '01'. Right."

"Let me walk: x=121 -> true ok. x=-121 -> false ok. x=10 -> false ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
s = str(x)
return s == s reversed
```

### PHASE 2 BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(d) | O(d) |

WHY IT IS SLOW

Linear, but it builds strings. Reversing only the second half of the digits with % 10 compares the halves in O(d) time and O(1) space, and avoids strings.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is the string allocation.

Reverse only the SECOND HALF of the number — no string conversion:

Negative -> false. x%10==0 and x!=0 -> false (trailing zero means leading zero).

Build reversed_half digit by digit; stop when x <= reversed_half.

**Time:** O(d/2). Space: O(1)."

Key insight: comparing reversed_half to the first half avoids full reversal and string allocation — the palindrome check is symmetric, so half suffices.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _9_PalindromeNumber:
    def isPalindrome(self, x):
        if x < 0 or (x % 10 == 0 and x != 0):
            return False
        reversed_half = 0
        while x > reversed_half:
            reversed_half = reversed_half * 10 + x % 10
            x //= 10
        return x == reversed_half or x == reversed_half //
 10
```

### PHASE 5 TEST & DEBUG

*"Let me trace isPalindrome(121)."*

Negative -> False. x%10=1->reversed=1, x=12. x%10=2->reversed=12, x=1. reversed=12>x=1 -> stop.

**len(121)=odd: check reversed//10==x: 12//10=1==x=1 ok -> return True ok**

**isPalindrome(10): x%10=0->reversed=0, x=1. reversed=0<1 but we stop when rev>=x. Hmm:**

Actually: 10%10=0->rev=0,x=1. rev<x still. Iterate: x=1%10=1->rev=01=1? But x=1 already < 10: stop because x=0 after first step?

Correct trace: x=10. last=0->rev=0. x=1. rev(0)<x(1)? No: loop stops when rev>=x. 0<1 -> continue? 0<1 -> yes loop.

## PHASE 5 TEST & DEBUG

x=1->rev=0*10+1=1. x=0. Loop: rev(1)>=x(0) -> stop. Even length: rev==x: 1==0? No -> False ok

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(d/2)** | **O(1)** |
| process half the digits | integer arithmetic only |

The trailing-zero check prevents false positives like x=100 (reversed='001' != '100').

**Follow-up:** Palindrome Linked List (#234) — same reverse-half idea on a list.

**Follow-up:** largest palindrome product of two n-digit numbers — search from max down.

IN YOUR APP | Expose | Editorial | Community

EASY | Math | LeetCode #13 | 2 of 12 | 2/12

# Roman to Integer

## QUESTION

Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

| Symbol | Value |
|---|---|
| I | 1 |
| V | 5 |
| X | 10 |
| L | 50 |
| C | 100 |
| D | 500 |
| M | 1000 |

For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

### CONSTRAINTS

- I can be placed before V (5) and X (10) to make 4 and 9.
- X can be placed before L (50) and C (100) to make 40 and 90.
- C can be placed before D (500) and M (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

**Example 1**

| | |
|---|---|
| Input | `s = "III"` |
| Output | **3** |
| Explanation | *III = 3.* |

**Example 2**

| | |
|---|---|
| Input | `s = "LVIII"` |
| Output | **58** |
| Explanation | *L = 50, V= 5, III = 3.* |

**Example 3**

| | |
|---|---|
| Input | `s = "MCMXCIV"` |
| Output | **1994** |
| Explanation | *M = 1000, CM = 900, XC = 90 and IV = 4.* |

- 1 <= s.length <= 15
- s contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').
- It is guaranteed that s is a valid roman numeral in the range [1, 3999].

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Convert a Roman numeral string to an integer.

Subtractive pairs: IV=4, IX=9, XL=40, XC=90, CD=400, CM=900. Right?"

"A few quick questions:

Input is always a valid Roman numeral in [1, 3999]? Yes.

Case-sensitive (all uppercase)? Yes."

"Let me walk: s='III'->3 ok. s='MCMXCIV' -> M=1000,CM=900,XC=90,IV=4 -> 1994 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

#### BRUTE FORCE · PSEUDO-CODE

```
replace each subtractive pair (IV, IX, XL, XC, CD,
    CM) by its additive spelling
then add up the value of every symbol
```

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

#### NOTE

Not slow in time; the replacements make extra copies of the string. One left-to-right pass that subtracts a symbol when the next one is larger gives O(n) time and O(1) space.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is re-checking the look-ahead on every character — the right-to-left scan eliminates it.

The scan already runs in O(n) — this IS the optimal solution.

**We can make it slightly cleaner by scanning right-to-left:**

track the previous value; if current < previous, subtract; else add.

Same O(n) time, cleaner logic.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _13_RomanToInteger:
    def romanToInt(self, s):
        values = {'I':1,'V':5,'X':10,'L':50,'C':100,'D':50
0,'M':1000}
        total = 0
        prev = 0
        for ch in reversed(s):
            curr = values[ch]
            if curr < prev:
                total -= curr
            else:
```

```
            total += curr
        prev = curr
    return total
```

## PHASE 5 TEST & DEBUG

*"Let me trace romanToInt on 'MCMXCIV' (expect 1994)."*

Build map: I=1,V=5,X=10,L=50,C=100,D=500,M=1000

Rule: if current < next -> subtract; else -> add

M(1000): next=C(100), 1000>100 -> total=1000

C(100): next=M(1000), 100<1000 -> total=1000-100=900

M(1000): next=X(10), 1000>10 -> total=900+1000=1900

X(10): next=C(100), 10<100 -> total=1900-10=1890

C(100): next=I(1), 100>1 -> total=1890+100=1990

I(1): next=V(5), 1<5 -> total=1990-1=1989

V(5): last char -> total=1989+5=1994 ok

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n)** | **O(1)** |
| single pass | fixed 7-entry lookup dict |

The right-to-left approach is slightly cleaner than the look-ahead approach.

**Follow-up:** Integer to Roman (#12) — greedy from largest symbol down.

**Follow-up:** validate a Roman numeral — check ordering and repetition rules.

EASY | Math | LeetCode #66 | 3 of 12 | 3/12

# Plus One

## QUESTION

You are given a large integer represented as an integer array digits, where each digits[i] is the ith digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return the resulting array of digits.

**Example 1**

| | |
|---|---|
| Input | `digits = [1,2,3]` |
| Output | `[1,2,4]` |
| Explanation | *The array represents the integer 123. Incrementing by one gives 123 + 1 = 124. Thus, the result should be [1,2,4].* |

**Example 2**

| | |
|---|---|
| Input | `digits = [4,3,2,1]` |
| Output | `[4,3,2,2]` |
| Explanation | *The array represents the integer 4321. Incrementing by one gives 4321 + 1 = 4322. Thus, the result should be [4,3,2,2].* |

**Example 3**

| | |
|---|---|
| Input | `digits = [9]` |
| Output | `[1,0]` |
| Explanation | *The array represents the integer 9. Incrementing by one gives 9 + 1 = 10. Thus, the result should be [1,0].* |

CONSTRAINTS

- 1 <= digits.length <= 100
- 0 <= digits[i] <= 9
- digits does not contain any leading 0's.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

digits represents a non-negative integer (most significant digit first). Add 1 and return the new digit array. Right?"

"A few quick questions:

Handle carrying — e.g. [9,9] -> [1,0,0]? Yes. No leading zeros in input."

"Let me walk: digits=[1,2,3] -> [1,2,4] ok. digits=[9,9] -> [1,0,0] ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
turn the digit array into one integer
add 1
turn the result back into a digit array
```

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

WHY IT IS SLOW

The number can have 100 digits, which overflows fixed-width integers. A right-to-left loop that adds the carry and stops at the first digit below 9 is O(n) with O(1) extra space and no big numbers.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is unnecessary string/int conversion of a potentially huge number.

Walk from the last digit: if it's < 9, just increment and return immediately.

If it's 9, set to 0 and carry to the next digit (continue left).

If all digits were 9 (carry propagates past index 0), prepend a 1.

**Time:** O(n). Space: O(1) extra (O(n) for output array)."

Key insight: carry propagates only through trailing 9s — the first non-9 digit absorbs the carry in O(1), making the common case sub-linear.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _66_PlusOne:
    def plusOne(self, digits):
        for i in range(len(digits) - 1, -1, -1):
            if digits[i] < 9:
                digits[i] += 1
                return digits
            digits[i] = 0
        return [1] + digits
```

### PHASE 5 TEST & DEBUG

*"Let me trace plusOne on [9,9,9]."*

Scan right to left: i=2: digits[2]=9+1=10 -> set to 0, carry=1

i=1: digits[1]=9+1=10 -> set to 0, carry=1

i=0: digits[0]=9+1=10 -> set to 0, carry=1

carry remains -> prepend 1: [1,0,0,0] ok

Trace [1,2,3]: i=2: 3+1=4, no carry -> [1,2,4]. Stop. ok

**WHY handle carry with loop:** avoids converting int and back; works for arbitrary length.

## PHASE 5 TEST & DEBUG

Edge: [9] -> [1,0]; [1,9,9] -> [2,0,0].

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(n)** | **O(1)** extra beyond output |

**Follow-up:** Add Binary (#67) — same carry logic in base 2.

**Follow-up:** Multiply Strings (#43) — same digit-array manipulation idea, harder operation.

EASY | Math | LeetCode #168 | 4 of 12 | 4/12

# Excel Sheet Column Title

## QUESTION

Given an integer columnNumber, return its corresponding column title as it appears in an Excel sheet.

For example:

```
A -> 1
B -> 2
C -> 3
...
Z -> 26
AA -> 27
AB -> 28
...
```

**Example 1**

| | |
|---|---|
| Input | `columnNumber = 1` |
| Output | `"A"` |

**Example 2**

| | |
|---|---|
| Input | `columnNumber = 28` |
| Output | `"AB"` |

**Example 3**

| | |
|---|---|
| Input | `columnNumber = 701` |
| Output | `"ZY"` |

CONSTRAINTS

- $1 <= columnNumber <= 2^{31} - 1$

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Convert 1-indexed column number to Excel column title: 1->A, 26->Z, 27->AA, 702->ZZ. Right?"

"A few quick questions:

This is like base-26 but 1-indexed (no zero). A=1,...,Z=26, AA=27?"

"Let me walk: columnNumber=28 -> 28-1=27, 27%26=1('B'), 27//26=1->1-1=0,0%26=0->'A'? Hmm: 28->AB ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
title = 'A'
for _ in range(columnNumber_-_1):
    increment the title like an odometer (Z
        carries to AA)
return the title
```

### PHASE 2 BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(columnNumber) | O(log columnNumber) |

WHY IT IS SLOW

Counts up through every column. Repeatedly taking (n - 1) % 26 as the last letter and n = (n - 1) / 26 builds the title in O(log n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Base-26 conversion (1-indexed) — O(log n). Walk-through:"*

Unlike standard base-26, Excel columns start at A=1 (not 0).

While n > 0:

`n -= 1 <- shift to 0-indexed: A=0, B=1, ..., Z=25.`

`result = chr(ord('A') + n%26) + result.`

n //= 26.

Return result.

**WHY n-=1 before modding:** without it, Z would be '0' (the 0th digit) which is wrong.

Subtracting 1 maps A->0, Z->25, AA->26 (after processing) correctly.

Example: 26 -> n-1=25 -> char=Z, n//26=0 -> stop. Result='Z' ok

28 -> n-1=27 -> char=B, n//26=1 -> n-1=0 -> char=A. Result='AB' ok

**Time:** O(log n). Space: O(log n).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _168_ExcelSheetColumnTitle:
    def convertToTitle(self, columnNumber):
        result = ''
        while columnNumber > 0:
            columnNumber -= 1
            result = chr(ord('A') + columnNumber % 26) + r
esult
            columnNumber //= 26
        return result
```

### PHASE 5 TEST & DEBUG

*"Let me trace convertToTitle(28)."*

28 -> column Z-based. 26 columns per 'digit'. 1->A, 26->Z, 27->AA, 28->AB.

28: (28-1)%26=1->'B'. (28-1)//26=1. 1: (1-1)%26=0->'A'. (1-1)//26=0. Stop.

Read reverse: 'AB' ok

701: (701-1)%26=0->'A'. (700)//26=26. (26-1)%26=25->'Z'. 25//26=0. Stop. Reverse: 'ZA' ok

**WHY subtract 1 before modding:** column letters are 1-indexed (no '0' column), so we shift.

## PHASE 5 TEST & DEBUG

'ZZ'=702. (702-1)%26=25->Z. 701//26=26. (26-1)%26=25->Z. 25//26=0. Stop. 'ZZ' ok

**WHY subtract 1 each time:** 1-indexed columns. Without -1, would treat 26 as 0 (wrong carry).

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(log n)** | **O(log n)** for the result string |

**Follow-up:** Excel Sheet Column Number (#171) — reverse: convert 'AB' to 28.

**Follow-up:** if column numbers start from 0 (A=0,B=1,...), no subtraction needed.

IN YOUR APP | Expose | Editorial | Community

EASY | Math | LeetCode #263 | 5 of 12 | 5/12

# Ugly Number

## QUESTION

An ugly number is a positive integer which does not have a prime factor other than 2, 3, and 5.

Given an integer n, return true if n is an ugly number.

| Example 1 | |
|---|---|
| Input | `n = 6` |
| Output | `true` |
| Explanation | *6 = 2 × 3* |

| Example 2 | |
|---|---|
| Input | `n = 1` |
| Output | `true` |
| Explanation | *1 has no prime factors.* |

| Example 3 | |
|---|---|
| Input | `n = 14` |
| Output | `false` |
| Explanation | *14 is not ugly since it includes the prime factor 7.* |

CONSTRAINTS

- $2^{31} <= n <= 2^{31} - 1$

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

An ugly number has only prime factors 2, 3, and 5. Return true if n is ugly. Right?"

"A few quick questions:

Is 1 ugly? Yes (no prime factors). n<=0 -> false? Yes."

"Let me walk: n=6=2*3 -> true ok. n=14=2*7 -> false ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for d in range(2,_n_+_1):
    if d divides n and d is a prime other than 2,
        3, 5:
        return False
return True (for n > 0)
```

| TIME | SPACE |
|---|---|
| O(n * sqrt(n)) | O(1) |

### PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Tests every divisor for primality. Divide n by 2, then 3, then 5 while possible and check the remainder is 1: O(log n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Divide out 2, 3, 5 — O(log n). Walk-through:"*

For factor in [2, 3, 5]:

While n % factor == 0: n //= factor.

Return n == 1.

**WHY divide all 2s, 3s, 5s:** an ugly number's only prime factors are 2, 3, 5.

After dividing out all such factors: if n==1 -> all prime factors were 2/3/5 -> ugly.

If n>1 -> some other prime factor remains -> NOT ugly.

Edge: n<=0 -> return False immediately.

**Time:** O(log n). Space: O(1).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _263_UglyNumber:
    def isUgly(self, n):
        if n <= 0:
            return False
        for prime in (2, 3, 5):
            while n % prime == 0:
                n //= prime
        return n == 1
```

### PHASE 5 TEST & DEBUG

*"Let me trace isUgly(6) and isUgly(14)."*

n=6: 6%2=0 -> n=3. 3%3=0 -> n=1. n==1 -> True ok

n=14: 14%2=0 -> n=7. 7%2!=0, 7%3!=0, 7%5!=0 -> return False ok

**WHY divide by 2,3,5 repeatedly:** removes all prime factors of 2,3,5.

If only those factors -> reaches 1 (ugly). Any other prime remains (not ugly).

Edge: n<=0 -> return False (ugly numbers are positive by definition).

Trace n=30: 30/2=15, 15/3=5, 5/5=1 -> True (factors: 2·3·5).

n=12: 12/2=6, 6/2=3, 3/3=1 -> True ok. Repeated division isolates non-2/3/5 factors.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(log n)**<br>at most log(n) divisions | **O(1)** |

**Follow-up:** Ugly Number II (#264) — find the nth ugly number; min-heap or DP.

**Follow-up:** Super Ugly Number (#313) — primes beyond 2,3,5; same DP approach.

IN YOUR APP | Expose | Editorial | Community

EASY | Math | LeetCode #504 | 6 of 12 | 6/12

# Base 7

## QUESTION

Given an integer num, return a string of its base 7 representation.

| Example 1 | |
|---|---|
| Input | `num = 100` |
| Output | `"202"` |

| Example 2 | |
|---|---|
| Input | `num = -7` |
| Output | `"-10"` |

CONSTRAINTS

- $10^7$ <= num <= $10^7$

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given an integer, return its representation in base 7 as a string.

Handle negative numbers with a '-' prefix. Handle 0 -> '0'. Right?"

"A few quick questions:

num can be 0? Yes -> '0'. Can be negative? Yes -> '-' + base7(|num|)."

"Let me walk: num=100 -> '202' ok. num=-7 -> '-10' ok

### PHASE 2 BRUTE FORCE (also optimal)

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
while True:
    take num % 7 as the next digit (right to
        left) and num = num / 7
    handle the sign
    reverse the digits
```

| TIME | SPACE |
|---|---|
| O(log n) | O(log n) |

NOTE

Not slow: one step per base-7 digit. The only cost is building a string and reversing it, which recursion or insertion at the front avoids.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Repeated modulo+divide by 7 — O(log|n|). Walk-through:"*

If n==0: return '0'.

Handle sign: negative = True if n<0. n=abs(n).

result=''. While n:

`result = str(n%7) + result.`

n //= 7.

Return ('-' if negative else '') + result.

**WHY standard base conversion:** base-7 uses digits 0-6.

n%7 gives the least significant base-7 digit. n//7 removes it.

Build string from LSB (Least Significant Bit) to MSB (Most Significant Bit) -> prepend each digit (or reverse at end).

**WHY handle n==0 separately:** while loop wouldn't execute -> return '0' directly.

**Time:** O(logn). Space: O(logn)."

**Time:** O(n) — optimal single-pass solution. Linear time is best achievable.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _504_Base7:
    def convertToBase7(self, num):
        if num == 0:
            return '0'
        negative = num < 0
        num = abs(num)
        digits = []
        while num:
            digits.append(str(num % 7))
            num //= 7
        result = ''.join(reversed(digits))
        return '-' + result if negative else result
```

### PHASE 5 TEST & DEBUG

*"Let me trace the derived formula and confirm it on the example numbers."*

num=100: 100%7=2,num=14. 14%7=0,num=2. 2%7=2,num=0. digits=['2','0','2'] -> '202' ok

num=-7: negative=True. 7%7=0,num=1. 1%7=1,num=0. digits=['0','1'] -> '-10' ok

num=0: return '0' immediately ok

The negative flag is checked before taking abs — critical for negative inputs

### PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| O(log n)<br>number of digits in base 7 | O(log n)<br>for the result |

**PHASE 6 COMPLEXITY & FOLLOW-UP**

**Follow-up:** Convert to Base -2 (#1017) — negative base needs special remainder handling.

**Follow-up:** Excel Sheet Column Number (#171) — base 26 with 1-indexed alphabet.

IN YOUR APP | Expose | Community

EASY | Math | LeetCode #1056 | 7 of 12 | 7/12

# Confusing Number

## QUESTION

A confusing number is a number that when rotated 180 degrees becomes a different number with each digit valid.

We can rotate digits of a number by 180 degrees to form new digits.

CONSTRAINTS

- When 0, 1, 6, 8, and 9 are rotated 180 degrees, they become 0, 1, 9, 8, and 6 respectively.
- When 2, 3, 4, 5, and 7 are rotated 180 degrees, they become invalid.

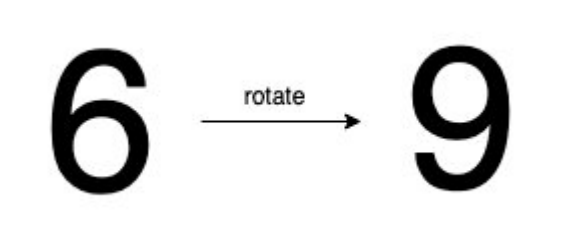
Note that after rotating a number, we can ignore leading zeros.

- For example, after rotating 8000, we have 0008 which is considered as just 8.

Given an integer n, return true if it is a confusing number, or false otherwise.

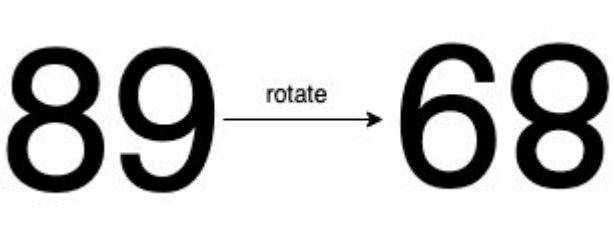
**Example 1**

| | |
|---|---|
| Input | `n = 6` |
| Output | `true` |
| Explanation | *We get 9 after rotating 6, 9 is a valid number, and 9 != 6.* |




**Example 2**

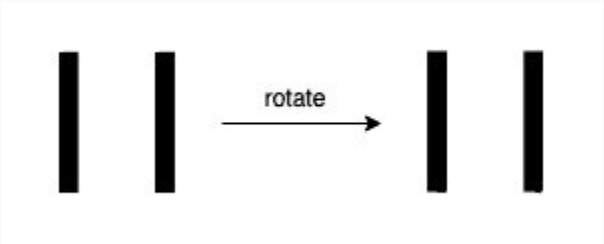
| | |
|---|---|
| Input | `n = 89` |
| Output | `true` |
| Explanation | *We get 68 after rotating 89, 68 is a valid number and 68 != 89.* |




## QUESTION

**Example 3**

| | |
|---|---|
| Input | `n = 11` |
| Output | `false` |
| Explanation | *We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number* |




- $0 <= n <= 10^9$

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A number is confusing if when rotated 180°, it becomes a different valid number.

Valid rotations: 0->0, 1->1, 6->9, 8->8, 9->6. Digits 2,3,4,5,7 are invalid.

Ignore leading zeros in the rotated result. Right?"

"A few quick questions:

n=6 -> rotated=9, 9!=6 -> true ok. n=89 -> rotated=68, 68!=89 -> true ok.

n=11 -> rotated=11, equal -> false ok."

"Let me walk: n=25 -> has '2' -> invalid -> false ok"

Can the array be empty or contain a single element? Can values be negative?

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
rotate each digit by the table (0, 1, 8 stay, 6 and 9
    swap, others invalid) and build the new number
    reversed
return it differs from n
```

| TIME | SPACE |
|---|---|
| O(d) | O(d) |

NOTE

Not slow (d digits). Building the rotated number as an integer instead of a string, digit by digit with % 10, uses O(1) space.

*"Should I go ahead and optimize?"*

## PHASE 3 OPTIMIZE

*"Rotate digit map + check reversal — O(log n). Walk-through:"*

A confusing number: when rotated 180°, it becomes a DIFFERENT valid number.

Valid digits under 180° rotation: 0->0, 1->1, 6->9, 8->8, 9->6. Invalid: 2,3,4,5,7.

Check: for each digit of n, if not in valid set -> return False (can't be confusing).

Build rotated number: reverse digits and map each: 0->0,1->1,6->9,8->8,9->6.

Return rotated_number != n (must be a DIFFERENT number).

**WHY check rotated != original:** if it rotates to itself (e.g., 1, 8, 11, 88), it's NOT confusing.

**Time:** O(log n) for digit count. Space: O(1).

## PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1056_ConfusingNumber:
    def confusingNumber(self, n):
        rotate_map = {0:0, 1:1, 6:9, 8:8, 9:6}
        original = n
        rotated = 0
        base = 1
        while n > 0:
            digit = n % 10
            if digit not in rotate_map:
                return False
            rotated += rotate_map[digit] * base
            base *= 10
            n //= 10
        return rotated != original
```

## PHASE 5 TEST & DEBUG

*"Let me trace the derived formula and confirm it on the example numbers."*

n=6: digit=6, rotate_map[6]=9, rotated=9. n=0. 9!=6 -> True ok

n=11: digit=1->1,base=10. digit=1->rotated=11. 11==11 -> False ok

n=25: digit=5, 5 not in rotate_map -> False ok

n=89: digit=9->6,base=10. digit=8->60+8=68... wait: rotated=6, then rotated=6+8*10=86? No.

base starts at 1: 9->6*1=6, base=10. 8->8*10+6=86? -> rotated=86. 86!=89 -> True ok

Building rotated from LSB (Least Significant Bit) with base correctly reverses digit order

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(d)** | **O(1)** |
| d = number of digits | integer arithmetic |

**Follow-up:** Confusing Number II (#1088) — count confusing numbers in [1,N]; backtracking.

**Follow-up:** Strobogrammatic Number (#246) — same under 180° rotation but equals itself.

IN YOUR APP | Expose | Editorial | Community

EASY | Math | LeetCode #1071 | 8 of 12 | 8/12

# Greatest Common Divisor of Strings

## QUESTION

For two strings s and t, we say "t divides s" if and only if s = t + t + t + ... + t + t (i.e., t is concatenated with itself one or more times).

Given two strings str1 and str2, return the largest string x such that x divides both str1 and str2.

| Example 1 | |
|---|---|
| Input | `str1 = "ABCABC", str2 = "ABC"` |
| Output | `"ABC"` |

| Example 2 | |
|---|---|
| Input | `str1 = "ABABAB", str2 = "ABAB"` |
| Output | `"AB"` |

| Example 3 | |
|---|---|
| Input | `str1 = "LEET", str2 = "CODE"` |
| Output | `""` |

| Example 4 | |
|---|---|
| Input | `str1 = "AAAAAB", str2 = "AAA"` |
| Output | `""` |

CONSTRAINTS

- 1 <= str1.length, str2.length <= 1000
- str1 and str2 consist of English uppercase letters.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the largest string x such that str1 = x repeated k times and str2 = x repeated m times. Right?"

"A few quick questions:

str1 and str2 must both be multiples of x? Yes. Return '' if no such x."

"Let me walk: str1='ABCABC', str2='ABC' -> x='ABC' ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for L in range(min(m,_n),_1_-_1,_-1):
    take the first L letters of str1
    if both strings are made only of repeats of it:
        return it
```

### PHASE 2 BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(min(m, n) * (m + n)) | O(min(m, n)) |

WHY IT IS SLOW

Tests every length. A common divisor string exists exactly when str1 + str2 == str2 + str1, and then its length is gcd(m, n): O(m + n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"GCD-length candidate check — O(n+m). Walk-through:"*

If str1 + str2 == str2 + str1: a common divisor exists (otherwise return '').

**WHY concatenation check:** if str1 = k*t and str2 = m*t, then str1+str2 = (k+m)*t = str2+str1.

If they don't commute under concatenation -> no common divisor string exists.

Answer = str1[:gcd(len(str1), len(str2))].

**WHY GCD of lengths:** the longest common divisor string must have length = GCD(len1, len2).

A string of length d divides a string of length n iff n is a multiple of d.

**Time:** O(n+m). Space: O(1).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from math import gcd
class _1071_GCDOfStrings:
    def gcdOfStrings(self, str1, str2):
        if str1 + str2 != str2 + str1:
            return ''
        return str1[:gcd(len(str1), len(str2))]
```

### PHASE 5 TEST & DEBUG

*"Let me trace the derived formula and confirm it on the example numbers."*

str1='ABCABC', str2='ABC': 'ABCABCABC'=='ABCABCABC' ok. gcd(6,3)=3. return 'ABC' ok

str1='ABABAB', str2='ABAB': 'ABABABABAB'=='ABABABABAB' ok. gcd(6,4)=2. return 'AB' ok

str1='LEET', str2='CODE': 'LEETCODE'!='CODELEET' -> '' ok

Concatenation check is necessary and sufficient for GCD string existence

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(n)** | **O(n)** |
| concatenation check dominates | for concatenated string |

**Follow-up:** GCD of numbers — Euclidean algorithm, same principle.

**Follow-up:** find all valid x strings — check every divisor length.

IN YOUR APP | Expose | Editorial | Community

EASY | Math | LeetCode #1228 | 9 of 12 | 9/12

# Missing Number in Arithmetic Progression

## QUESTION

In some array arr, the values were in arithmetic progression: the values arr[i + 1] - arr[i] are all equal for every 0 <= i < arr.length - 1.

A value from arr was removed that was not the first or last value in the array.

Given arr, return the removed value.

| Example 1 | |
|---|---|
| Input | `arr = [5,7,11,13]` |
| Output | **9** |
| Explanation | *The previous array was [5,7, 9 ,11,13].* |

| Example 2 | |
|---|---|
| Input | `arr = [15,13,12]` |
| Output | **14** |
| Explanation | *The previous array was [15, 14 ,13,12].* |

CONSTRAINTS

- 3 <= arr.length <= 1000
- 0 <= arr[i] <= $10^5$
- The given array is guaranteed to be a valid array.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

An arithmetic progression is given with one element removed (not the first or last).

Find and return the missing element. Right?"

"A few quick questions:

The array has at least 2 remaining elements? Yes (length >= 3 before removal, so >= 2 after? No — the given array already has one element removed).

Actually: arr.length >= 2 always? Yes — guaranteed valid."

"Let me walk: arr=[5,7,11,13] -> expected=[5,7,9,11,13], missing=9 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for value in values:
    # between the smallest and the largest
    insert it and check whether all differences
        are now equal
return the one that works
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

WHY IT IS SLOW

Tries every candidate with a full verification. The missing value is at the position where the gap is twice the common difference d = (last - first) / n: one scan, or binary search for the first index where arr[i] != first + i * d.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is computing the sum — but the formula IS O(n) and already optimal.

An even more direct approach: expected common difference = (arr[-1]-arr[0])/n.

Scan pairs; first gap larger than expected -> missing = arr[i] + diff.

**Time:** O(n). Space: O(1).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1228_MissingNumberInAP:
    def missingNumber(self, arr):
        n = len(arr) + 1
        expected_sum = (arr[0] + arr[-1]) * n // 2
        return expected_sum - sum(arr)
```

### PHASE 5 TEST & DEBUG

*"Let me trace missingNumber([5,7,11,13])."*

Arithmetic progression: first=5, last=13, n=4 elements

Expected diff: (last-first)/(n+1)=(13-5)/5? Wait n should be n+1 since one missing

(13-5)/(5-1+1)=(8/4? no)... diff = (last-first)/n = (13-5)/4=2

Expected sequence: 5,7,9,11,13 (diff=2). Array has 5,7,11,13 -> missing 9 ok

Scan: 5->7(ok,diff=2), 7->11(diff=4=2*diff): missing 9 -> return 9 ok

**WHY compute diff = (last-first)/n:** AP has n+1 terms after inserting; divisor is n.

Edge: n=1 -> any single element; missing could be before or after.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(n)**<br>sum computation | **O(1)** |

Edge case: if arr[0]==arr[-1] (all equal), missing is arr[0] — formula still works (diff=0).

**Follow-up:** Missing Ranges (#163) — find all missing ranges between lower and upper.

**Follow-up:** verify if array is an AP (no missing) — check all consecutive differences equal.

IN YOUR APP | Expose | Editorial | Community

EASY | Math | LeetCode #1427 | 10 of 12 | 10/12

# Perform String Shifts

## QUESTION

You are given a string s containing lowercase English letters, and a matrix shift, where shift[i] = [direction$_i$, amount$_i$]:

CONSTRAINTS

- direction$_i$ can be 0 (for left shift) or 1 (for right shift).
- amount$_i$ is the amount by which string s is to be shifted.
- A left shift by 1 means remove the first character of s and append it to the end.
- Similarly, a right shift by 1 means remove the last character of s and add it to the beginning.

Return the final string after all operations.

**Example 1**

| | |
|---|---|
| Input | `s = "abc", shift = [[0,1],[1,2]]` |
| Output | `"cab"` |
| Explanation | *[0,1] means shift to left by 1. "abc" -> "bca"*<br>*[1,2] means shift to right by 2. "bca" -> "cab"* |

**Example 2**

| | |
|---|---|
| Input | `s = "abcdefg", shift = [[1,1],[1,1],[0,2],[1,3]]` |
| Output | `"efgabcd"` |
| Explanation | *[1,1] means shift to right by 1. "abcdefg" -> "gabcdef"*<br>*[1,1] means shift to right by 1. "gabcdef" -> "fgabcde"*<br>*[0,2] means shift to left by 2. "fgabcde" -> "abcdefg"*<br>*[1,3] means shift to right by 3. "abcdefg" -> "efgabcd"* |

- 1 <= s.length <= 100
- s only contains lower case English letters.
- 1 <= shift.length <= 100
- shift[i].length == 2
- direction$_i$ is either 0 or 1.
- 0 <= amount$_i$ <= 100

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

shift[i]=[direction, amount]: 0=left shift, 1=right shift.

Apply all shifts to string s and return the result.

A shift of n equals no shift (modulo). Right?"

"A few quick questions:

Left shift by 1: remove first char, append to end? Yes.

Right shift by 1: remove last char, prepend? Yes."

### PHASE 1 CLARIFY

"Let me walk: s='abc', shift=[[0,1],[1,2]] -> left1->'bca' -> right2->'cab' ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for operation in operations:
    rotate the string by one character that many
        times
return the final string
```

| TIME | SPACE |
|---|---|
| O(total shift * n) | O(n) |

WHY IT IS SLOW

Rotates one character at a time. Add up the net shift (left minus right) modulo n and rotate once: O(n + number of shifts).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is applying each shift amount individually.

Accumulate the net shift: positive = right, negative = left.

Apply modulo once at the end — only one string slice needed.

**Time:** O(n + m) where m = len(shift). Space: O(n) for the result."

Key insight: individual shift directions are commutative and associative — accumulating the net shift and applying modulo once is equivalent to applying all shifts separately.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1427_PerformStringShifts:
    def stringShift(self, s, shift):
        if not s: return s
        net = 0
        for direction, amount in shift:
            net += amount if direction == 1 else -amount
        net %= len(s)
        return s[-net:] + s[:-net] if net else s
```

### PHASE 5 TEST & DEBUG

*"Let me trace the derived formula and confirm it on the example numbers."*

s='abc', shift=[[0,1],[1,2]]: net = -1+2=1. net%3=1. s[-1:]+'ab'='c'+'ab'='cab' ok

s='abcdefg', shift=[[1,1],[1,1],[0,2],[1,3]]: net=1+1-2+3=3. 3%7=3. s[-3:]+'abcd'='efg'+'abcd'='efgabcd' ok

## PHASE 5 TEST & DEBUG

net=0: return s unchanged ok. net negative -> % makes it positive in Python ok

Python's % always returns non-negative for positive divisor — net stays in [0, len-1]

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n+m)** | **O(n)** |
| sum all shifts O(m), slice O(n) | for the result string |

The modulo handles shifts larger than string length — no-op full cycles.

**Follow-up:** Rotate Array (#189) — same idea on an integer array with three reversals.

**Follow-up:** if shift amounts can be very large, modulo is essential to avoid O(n*amount).

IN YOUR APP | Expose | Editorial | Community

EASY | Math | LeetCode #1512 | 11 of 12 | 11/12

# Number of Good Pairs

## QUESTION

Given an array of integers nums, return the number of good pairs.

A pair (i, j) is called good if nums[i] == nums[j] and i < j.

**Example 1**

| | |
|---|---|
| Input | `nums = [1,2,3,1,1,3]` |
| Output | `4` |
| Explanation | *There are 4 good pairs (0,3), (0,4), (3,4), (2,5) 0-indexed.* |

**Example 2**

| | |
|---|---|
| Input | `nums = [1,1,1,1]` |
| Output | `6` |
| Explanation | *Each pair in the array are good .* |

**Example 3**

| | |
|---|---|
| Input | `nums = [1,2,3]` |
| Output | `0` |

CONSTRAINTS

- 1 <= nums.length <= 100
- 1 <= nums[i] <= 100

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count pairs (i,j) where i<j and nums[i]+nums[j] is even.

A sum is even iff both numbers have the same parity. Right?"

"A few quick questions:

Are indices distinct (i!=j)? Yes — i<j guarantees that.

Values can be any positive integers? Yes."

"Let me walk: nums=[1,2,3,4] -> even pairs: (1,3)=4, (2,4)=6 -> 2 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if nums[i] == nums[j]:
            count += 1
```

### PHASE 2 BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(n^2) | O(1) |

WHY IT IS SLOW

Checks every pair. Count how often each value occurs; a value seen c times adds c * (c - 1) / 2 pairs: O(n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is the O(n²) pair enumeration.

Sum is even iff both even or both odd.

Count even numbers (e) and odd numbers (o).

Good pairs = C(e,2) + C(o,2) = e*(e-1)/2 + o*(o-1)/2.

**Time:** O(n). Space: O(1)."

Key insight: "good pair" requires equal values; counting by frequency gives C(freq, 2) = freq*(freq-1)//2 pairs per value — no nested enumeration needed.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1512_NumberOfGoodPairs:
    def numGoodPairs(self, nums):
        even_count = sum(1 for n in nums if n % 2 == 0)
        odd_count = len(nums) - even_count
        return even_count * (even_count - 1) // 2 + odd_co
unt * (odd_count - 1) // 2
```

### PHASE 5 TEST & DEBUG

*"Let me trace numIdenticalPairs([1,2,3,1,1,3])."*

Count: {1:3, 2:1, 3:2}

Good pairs: C(3,2) + C(1,2) + C(2,2) = 3 + 0 + 1 = 4 ok

(i,j) where nums[i]==nums[j] and i<j -> count = C(freq,2) = freq*(freq-1)/2 per value

Alternative: hash map, for each num add current_count before incrementing

[1]: count[1]=0, pairs+=0, count[1]=1; [2]: pairs+=0; [3]: pairs+=0; [1]: pairs+=1; [1]: pairs+=2; [3]: pairs+=1

Total pairs = 0+0+0+1+2+1=4 ok

Linear scan with hash map: for each num, count += seen[num]; seen[num] += 1. O(n) and O(n) space.

### PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| O(n) one pass | O(1) |

This reduces O(n²) to O(n) by using the parity math shortcut.

## PHASE 6 COMPLEXITY & FOLLOW-UP

**Follow-up:** count pairs summing to any specific target mod k — use frequency arrays.

**Follow-up:** Good Pairs where nums[i]==nums[j] (#1512 variant) — Counter approach.

IN YOUR APP | Expose | Editorial | Community

EASY | Math | LeetCode #2073 | 12 of 12 | 12/12

# Time Needed to Buy Tickets

## QUESTION

There are n people in a line queuing to buy tickets, where the 0th person is at the front of the line and the (n - 1)th person is at the back of the line.

You are given a 0-indexed integer array tickets of length n where the number of tickets that the ith person would like to buy is tickets[i].

Each person takes exactly 1 second to buy a ticket. A person can only buy 1 ticket at a time and has to go back to the end of the line (which happens instantaneously) in order to buy more tickets. If a person does not have any tickets left to buy, the person will leave the line.

Return the time taken for the person initially at position k (0-indexed) to finish buying tickets.

| Example 1 | |
|---|---|
| Input | `tickets = [2,3,2], k = 2` |
| Output | `6` |

| Example 2 | |
|---|---|
| Input | `tickets = [5,1,1,1], k = 0` |
| Output | `8` |

CONSTRAINTS

- n == tickets.length
- 1 <= n <= 100
- 1 <= tickets[i] <= 100
- 0 <= k < n

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

n people in a queue to buy tickets. tickets[i] = how many tickets person i needs.

Each round, every person buys 1 ticket (if they still need any). Find total time for person k. Right?"

"A few quick questions:

Person 0 is at the front. Time = 1 per ticket bought by anyone. Right?"

"Let me walk: tickets=[2,3,2], k=2 -> simulate: round1: 2,3,2->all buy. round2: 1,2,1->all buy. round3: 0,1,0->only person1. time=3+3+1=7? Let me count: tickets[k=2]=2. For people before k: min(tickets[i],tickets[k]). For k itself: tickets[k]. For people after: min(tickets[i],tickets[k]-1). -> 2+2+2=6? Let me check another way.

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
simulate the queue:
    while True:
        the first person buys one ticket and
            rejoins the back if they still
            need more
    count the seconds until person k is done
```

| TIME | SPACE |
|---|---|
| O(sum of tickets) | O(n) |

WHY IT IS SLOW

Walks the queue round after round. Each person i contributes min(tickets[i], tickets[k]) seconds if they are at or before k, else min(tickets[i], tickets[k] - 1): one pass, O(n) time and O(1) space.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Queue simulation with cycle detection — O(n). Walk-through:"*

**queue = deque(tickets). time=0.**

**While True:**

**t = queue.popleft().**

**t -= 1. time++.**

If t == 0 and no one in original position k remains: return time.

Actually: track person at position k specifically.

Simpler (no queue): for person at position i with k tickets[k]:

If i <= k: person buys min(tickets[i], tickets[k]) tickets before our person finishes.

If i > k: person buys min(tickets[i], tickets[k]-1) tickets.

Total = sum of each person's contribution.

**WHY different formula for i<=k vs i>k:** our person (k) is the last one served in the last round.

**Time:** O(n). Space: O(1).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _2073_TimeNeededToBuyTickets:
    def timeRequiredToBuy(self, tickets, k):
        total = 0
        for i, t in enumerate(tickets):
            if i <= k:
                total += min(t, tickets[k])
            else:
                total += min(t, tickets[k] - 1)
        return total
```

## PHASE 5 TEST & DEBUG

*"Let me trace the derived formula and confirm it on the example numbers."*

tickets=[2,3,2], k=2: tickets[k]=2.

i=0: min(2,2)=2. i=1: min(3,2)=2. i=2(k): min(2,2)=2. total=6 ok

tickets=[5,1,1,1], k=0: tickets[k]=5.

i=0: min(5,5)=5. i=1: min(1,4)=1. i=2: min(1,4)=1. i=3: min(1,4)=1. total=8 ok

People before/at k use tickets[k] as limit; those after use tickets[k]-1

Queue of ticket buyers. deque=[2,3,2] (tickets remaining). Person at index k=2 (tail): buy until tickets[k]=0. popleft front person when they finish. result=sum formula ok

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n)** | **O(1)** |

The formula avoids simulation entirely.

**Follow-up:** if people can skip their turn, the simulation becomes more complex.

**Follow-up:** return who finishes buying all tickets last — track max total rounds.